

3 Write the program to implement Data Pre-processing for Machine learning

```

import pandas as pd
import numpy as np
# Load data
data = pd.DataFrame({
    'age': [25, np.nan, 30, 45, np.nan],
    'salary': [50000, 60000, np.nan, 65000, 70000],
    'city': ['New York', 'Los Angeles', 'New York', 'San Francisco', np.nan],
    'target': [1, 0, 1, 0, 1]
})
# Display the original data
print("Original Data:")
print(data)

# Display missing values
print("Missing values:")
print(data.isnull().sum())
# Percentage of missing values
print("percentage of Missing values:")
print(data.isnull().mean() * 100)

# Handling missing values
data = data.dropna()
print('after removing rows')
print(data)
#data = data.dropna(axis=1)
#print('after removing columns')
#print(data)

from sklearn.impute import SimpleImputer
# For numerical features, use mean imputation
#num_features = ['age', 'salary']
#imputer_num = SimpleImputer(strategy='mean')
#data[num_features] = imputer_num.fit_transform(data[num_features])
# For categorical features, use the most frequent value imputation
#cat_features = ['city']
#imputer_cat = SimpleImputer(strategy='most_frequent')
#data[cat_features] = imputer_cat.fit_transform(data[cat_features])

from sklearn.preprocessing import StandardScaler, LabelEncoder
# Encoding categorical features
cat_features = ['city']
label_encoders = {}

```

```

for col in cat_features:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le
# Feature scaling
scaler = StandardScaler()
num_features = ['age', 'salary']
data[num_features] = scaler.fit_transform(data[num_features])

data.to_csv('cleaned_dataset.csv', index=False)
data1 = pd.read_csv('cleaned_dataset.csv')
print('Clean Data: in csv')
print(data1)

```

- 4 Write a program to implement Classification Algorithm. Calculate the accuracy, precision, recall and lift measure.

```

import numpy as np
import pandas as pd
# Load dataset
from sklearn.datasets import load_iris
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# Split data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=10)
knn.fit(X_train, y_train)
y_pred=knn.predict(X_test)

#Evaluate the model
from sklearn.metrics import classification_report, confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print("Confusion Matrix:\n", cm)
print("classification report", classification_report(y_test, y_pred))
#print("KNN_score=" ,knn.score(X_test, y_test))

#Validate the model on Unseen data
test_new=np.array([[4.5,1.2,3.1,2.1]])
pred=knn.predict(test_new)

```

```
print('predicted unseen data label')
if pred==0:
    print('Setosa')
elif pred==1:
    print('Versicolor')
else:
    print('Verginica')
```